



Retail Store Inventory Demand Forecasting using XGBoost + Prophet



Introduction

Retail businesses rely heavily on accurate demand forecasting to optimize inventory, pricing, and supply chain decisions.

In this notebook, we build a hybrid deep learning model combining a Transformer network and XGBoost, leveraging both temporal dependencies and tree-based feature learning.

This project demonstrates:

- ✓ Real-world feature engineering (lag, rolling stats, time features)
- ✓ Transformer architecture for temporal feature extraction
- ✓ Hybrid stacking with XGBoost for residual correction
- ✓ Clear evaluation and visualizations

Step 1: Import Libraries & Load Dataset

We start by importing all the required packages for:

Data preprocessing

Model building (TensorFlow, XGBoost, LightGBM)

Visualization (Matplotlib, Seaborn)

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import xgboost as xgb
6 import lightgbm as lgb
7 import tensorflow as tf
8 from tensorflow.keras.models import Sequential, Model
9 from tensorflow.keras.layers import LSTM, Dense, Dropout, Input, MultiHeadAttention, LayerNormalization, Add, Glo
10 from sklearn.preprocessing import MinMaxScaler
11 from sklearn.metrics import mean_absolute_percentage_error
12 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
13 from tensorflow.keras.optimizers import Adam
14 import pickle # Import pickle
15 import joblib # Import joblib
16
```

Step 2: Data Loading and Initial Exploration

We load the dataset and perform basic feature exploration.

```
1 # Load the dataset
2 df = pd.read_csv('/content/retail_store_inventory.csv')
3
```

```
1 df.head()
```

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Demand Forecast	Price	Discount	Weather Condition	Holiday/Prom
0	2022-01-01	S001	P0001	Groceries	North	231	127	55	135.47	33.50	20	Rainy	
1	2022-01-01	S001	P0002	Toys	South	204	150	66	144.04	63.01	20	Sunny	
2	2022-01-01	S001	P0003	Toys	West	102	65	51	74.02	27.99	10	Sunny	
3	2022-01-01	S001	P0004	Toys	North	469	61	164	62.18	32.72	10	Cloudy	
4	2022-01-01	S001	P0005	Electronics	East	166	14	135	9.26	73.64	0	Sunny	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 # Checking the shape and data types of the df
2
3 print(df.shape)
4 print(df.info())
```

```
(73100, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73100 entries, 0 to 73099
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                   73100 non-null object
1   Store ID               73100 non-null object
2   Product ID             73100 non-null object
3   Category               73100 non-null object
4   Region                 73100 non-null object
5   Inventory Level        73100 non-null int64
6   Units Sold             73100 non-null int64
7   Units Ordered          73100 non-null int64
8   Demand Forecast        73100 non-null float64
9   Price                  73100 non-null float64
10  Discount               73100 non-null int64
11  Weather Condition      73100 non-null object
12  Holiday/Promotion      73100 non-null int64
13  Competitor Pricing     73100 non-null float64
14  Seasonality            73100 non-null object
dtypes: float64(3), int64(5), object(7)
memory usage: 8.4+ MB
None
```

This dataset contains:

- Store & Product information
- Inventory and sales data
- Demand forecast values
- Temporal details (Date, Holiday, Discount, etc.)

Additionally, the dataset is currently complete with no missing values

✓ **Step 3: Feature Engineering**

We extract time-based features, generate lag features, and compute rolling statistics to capture historical patterns.

```
1 # --- Data Preprocessing and Feature Engineering ---
2 # Convert 'Date' column to datetime objects
3 df['Date'] = pd.to_datetime(df['Date'])
4
5 # Extract time-based features
6 df['year'] = df['Date'].dt.year # Extract the year from the date
7 df['month'] = df['Date'].dt.month # Extract the month from the date
8 df['day'] = df['Date'].dt.day # Extract the day from the date
9 df['dayofweek'] = df['Date'].dt.dayofweek # Extract the day of the week (Monday=0, Sunday=6)
10 df['weekofyear'] = df['Date'].dt.isocalendar().week.astype(int) # Extract the week of the year
11
```

```
12 # Sort the DataFrame by 'Date'
13 df = df.sort_values(by='Date').reset_index(drop=True)
```

↳ Lag & Rolling Features

Creating granular features using lag and rolling windows for multiple metrics such as Inventory Level, Units Sold, and Price.

```
1 # Create lag features
2 lag_period = 7 # Define the number of previous days to look back for lag features
3 for col in ['Inventory Level', 'Units Sold', 'Units Ordered', 'Demand Forecast', 'Price']:
4     # Create lag features for each specified column, grouped by Store ID and Product ID
5     df[f'{col}_lag_{lag_period}'] = df.groupby(['Store ID', 'Product ID'])[col].shift(lag_period)
6
7 # Create rolling window features (mean and std)
8 rolling_window = 7 # Define the size of the rolling window
9 for col in ['Inventory Level', 'Units Sold', 'Units Ordered', 'Demand Forecast', 'Price']:
10     # Calculate rolling mean for each specified column, grouped by Store ID and Product ID
11     df[f'{col}_rolling_mean_{rolling_window}'] = df.groupby(['Store ID', 'Product ID'])[col].rolling(window=roll
12     # Calculate rolling standard deviation for each specified column, grouped by Store ID and Product ID
13     df[f'{col}_rolling_std_{rolling_window}'] = df.groupby(['Store ID', 'Product ID'])[col].rolling(window=rolli
14
15 # Handle potential missing values created by lag and rolling window features (fill with 0)
16 df = df.fillna(0)
17
```

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73100 entries, 0 to 73099
Data columns (total 35 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Date                                     73100 non-null  datetime64[ns]
1   Store ID                                73100 non-null  object
2   Product ID                              73100 non-null  object
3   Category                                73100 non-null  object
4   Region                                  73100 non-null  object
5   Inventory Level                          73100 non-null  int64
6   Units Sold                              73100 non-null  int64
7   Units Ordered                           73100 non-null  int64
8   Demand Forecast                         73100 non-null  float64
9   Price                                   73100 non-null  float64
10  Discount                                73100 non-null  int64
11  Weather Condition                       73100 non-null  object
12  Holiday/Promotion                       73100 non-null  int64
13  Competitor Pricing                     73100 non-null  float64
14  Seasonality                             73100 non-null  object
15  year                                    73100 non-null  int32
16  month                                   73100 non-null  int32
17  day                                     73100 non-null  int32
18  dayofweek                              73100 non-null  int32
19  weekofyear                             73100 non-null  int64
20  Inventory Level_lag_7                   73100 non-null  float64
21  Units Sold_lag_7                        73100 non-null  float64
22  Units Ordered_lag_7                     73100 non-null  float64
23  Demand Forecast_lag_7                   73100 non-null  float64
24  Price_lag_7                             73100 non-null  float64
25  Inventory Level_rolling_mean_7          73100 non-null  float64
26  Inventory Level_rolling_std_7           73100 non-null  float64
27  Units Sold_rolling_mean_7               73100 non-null  float64
28  Units Sold_rolling_std_7                73100 non-null  float64
29  Units Ordered_rolling_mean_7            73100 non-null  float64
30  Units Ordered_rolling_std_7             73100 non-null  float64
31  Demand Forecast_rolling_mean_7          73100 non-null  float64
32  Demand Forecast_rolling_std_7           73100 non-null  float64
33  Price_rolling_mean_7                    73100 non-null  float64
34  Price_rolling_std_7                     73100 non-null  float64
dtypes: datetime64[ns](1), float64(18), int32(4), int64(6), object(6)
memory usage: 18.4+ MB
```

*This ensures the model sees past behavior trends for each store and product.**

↳ Evaluating XGBoost for Forecasting Units Sold

```

1 from sklearn.preprocessing import LabelEncoder
2
3 df_model = df.copy().sort_values("Date").reset_index(drop=True)
4
5 # Encode categorical columns
6 cat_cols = ["Store ID", "Product ID", "Category", "Region",
7            "Weather Condition", "Seasonality"]
8
9 le_dict = {}
10 for col in cat_cols:
11     le = LabelEncoder()
12     df_model[col] = le.fit_transform(df_model[col])
13     le_dict[col] = le # store encoders if needed later

```

```

1 # Splitting the data for training and testing
2
3 cutoff_date = df_model["Date"].max() - pd.Timedelta(days=90)
4
5 train = df_model[df_model["Date"] <= cutoff_date].copy()
6 test = df_model[df_model["Date"] > cutoff_date].copy()
7

```

```

1 # Creating our variables for modeling based on "Units Sold"
2
3 target = "Units Sold"
4
5 drop_cols = ["Date", target]
6
7 # X matrices
8 X_train = train.drop(columns=drop_cols)
9 X_test = test.drop(columns=drop_cols)
10
11 # y vectors
12 y_train = train[target]
13 y_test = test[target]
14

```

```

1 # Building and Training the XGBoost model
2
3 from xgboost import XGBRegressor
4
5 xgb_model = XGBRegressor(
6     n_estimators=500,
7     learning_rate=0.05,
8     max_depth=8,
9     subsample=0.8,
10    colsample_bytree=0.8,
11    objective="reg:squarederror",
12    tree_method="hist" # efficient
13 )
14
15 xgb_model.fit(X_train, y_train)
16

```

▼ XGBRegressor ⓘ ?

```

XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.8, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.05, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=8,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=500,
              n_jobs=None, num_parallel_tree=None, ...)

```

```

1 # Evaluating our model
2
3 from sklearn.metrics import mean_squared_error, mean_absolute_error
4 import numpy as np
5
6 y_pred = xgb_model.predict(X_test)
7

```



```

8 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
9 mae = mean_absolute_error(y_test, y_pred)
10
11 print("XGBoost RMSE:", rmse)
12 print("XGBoost MAE:", mae)
13

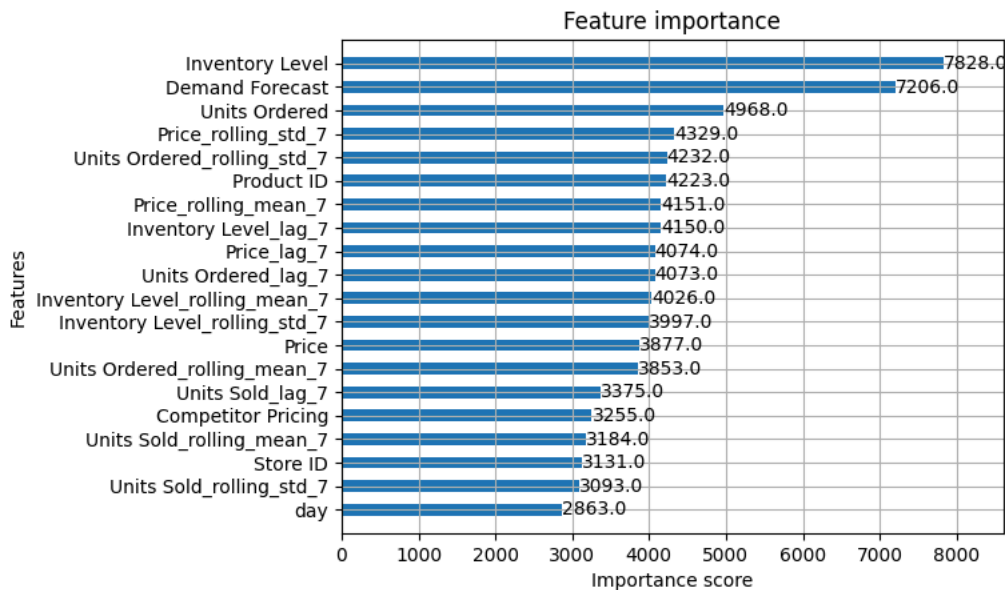
```

XGBoost RMSE: 8.76798427335345
XGBoost MAE: 7.416592121124268

```

1 # Observing our key features from the model
2
3 import matplotlib.pyplot as plt
4 import xgboost as xgb
5
6 xgb.plot_importance(xgb_model, max_num_features=20, height=0.5)
7 plt.show()
8

```



```

1 # Use only the correctly encoded dataframe
2 df_full = df_model.copy()
3
4 store_ids = df_full["Store ID"].unique()
5 product_ids = df_full["Product ID"].unique()
6
7 print("Stores:", store_ids)
8 print("Products:", product_ids)
9
10 for store in store_ids:
11     for product in product_ids:
12
13         # ---- Filter to store + product ----
14         df_pair = df_full[(df_full["Store ID"] == store) &
15                           (df_full["Product ID"] == product)].copy()
16
17         if df_pair.empty:
18             print(f"Skipping {store} {product} - no data")
19             continue
20
21         # Sort chronologically
22         df_pair = df_pair.sort_values("Date")
23
24         # ---- Time-based split ----
25         cutoff_date = df_pair["Date"].max() - pd.Timedelta(days=90)
26         train_pair = df_pair[df_pair["Date"] <= cutoff_date]
27         test_pair = df_pair[df_pair["Date"] > cutoff_date]
28
29         # Ensure we have both train and test points
30         if len(train_pair) == 0 or len(test_pair) == 0:
31             print(f"Skipping {store} {product} - insufficient data")
32             continue

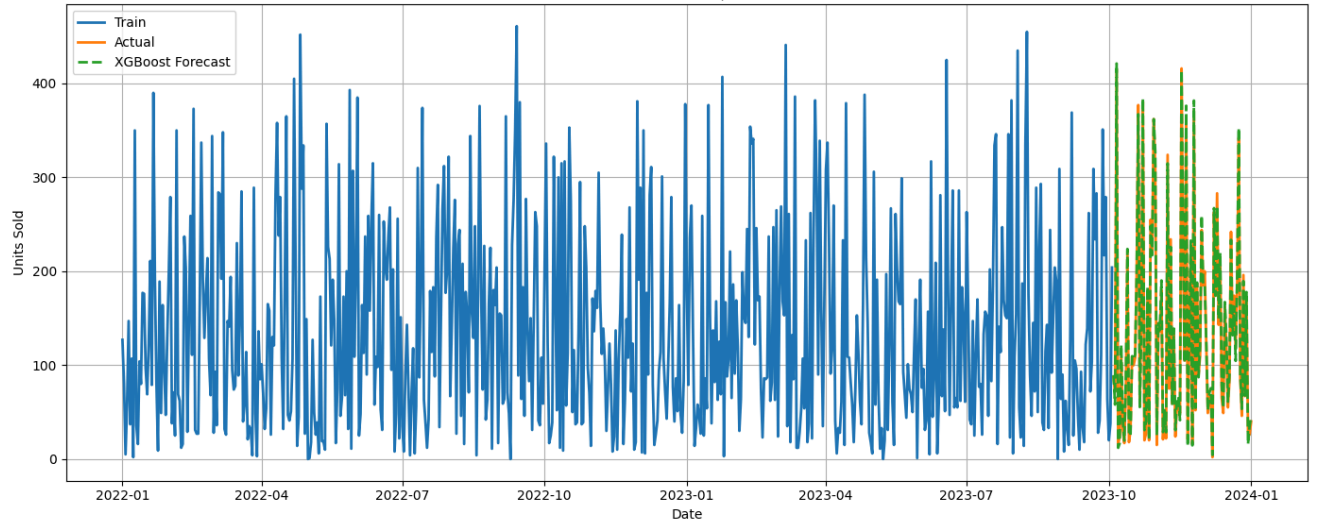
```

```
33
34 # ---- Extract X and y ----
35 drop_cols = ["Date", "Units Sold"]
36
37 X_train = train_pair.drop(columns=drop_cols)
38 X_test = test_pair.drop(columns=drop_cols)
39
40 y_train = train_pair["Units Sold"].values
41 y_test = test_pair["Units Sold"].values
42
43 # ---- Predict ----
44 y_pred = xgb_model.predict(X_test)
45
46 # ---- Plot ----
47 plt.figure(figsize=(14, 6))
48 plt.plot(train_pair["Date"], y_train, label="Train", linewidth=2)
49 plt.plot(test_pair["Date"], y_test, label="Actual", linewidth=2)
50 plt.plot(test_pair["Date"], y_pred, label="XGBoost Forecast", linestyle="--", linewidth=2)
51
52 plt.title(f"Units Sold: Store {store} | Product {product}")
53 plt.xlabel("Date")
54 plt.ylabel("Units Sold")
55 plt.legend()
56 plt.grid(True)
57 plt.tight_layout()
58
59 plt.show()
```

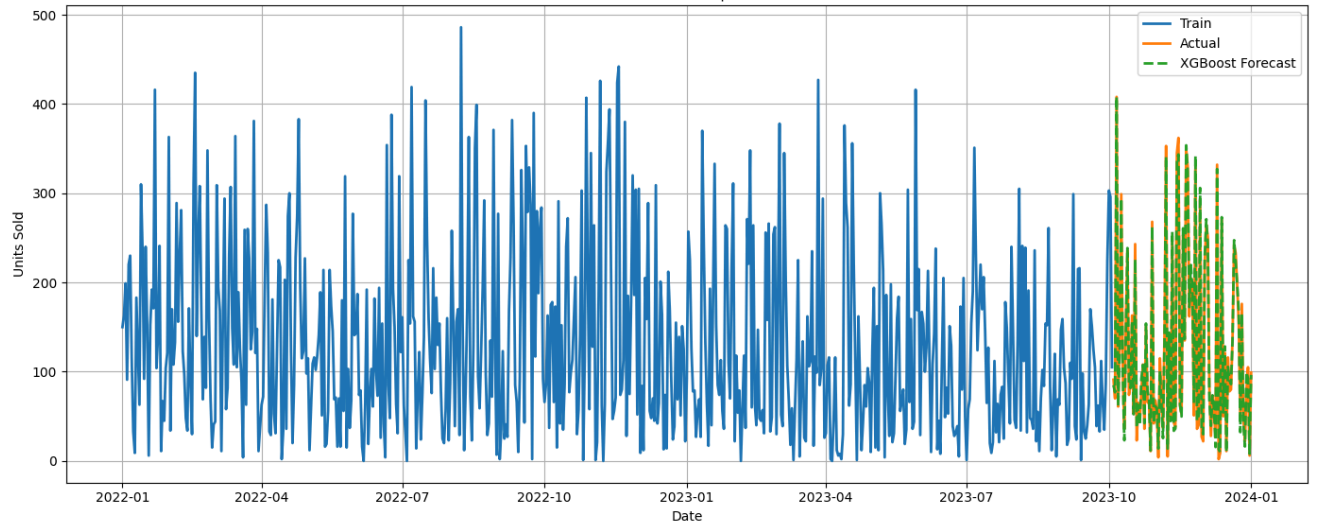

Stores: [0 1 2 3 4]

Products: [0 1 2 3 4 5 6 7 8 9 11 12 14 15 16 17 18 19 13 10]

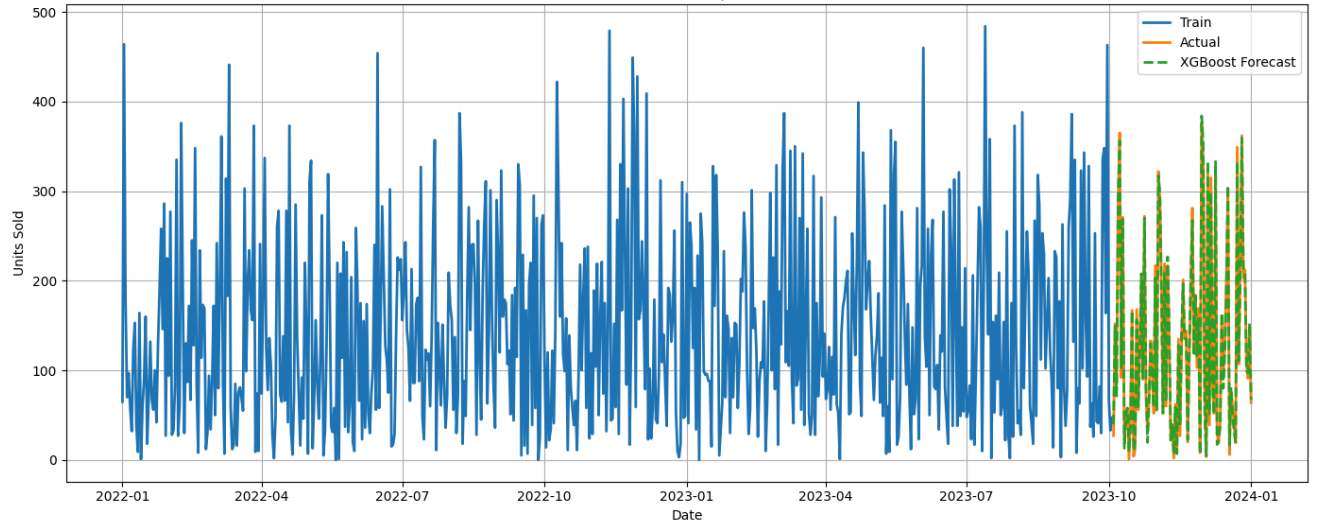
Units Sold: Store 0 | Product 0



Units Sold: Store 0 | Product 1



Units Sold: Store 0 | Product 2



Units Sold: Store 0 | Product 3

